UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 12

Torres de Hanoi en Arte ASCII

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

Estructura de la sesión

Tiempo	Bloque	Tipo
10 min	Historia: Lucas 1883, la leyenda del templo	Exposición
10 min	Las tres reglas y $M(n) = 2^n - 1$	Pizarrón
5 min	Arte ASCII como interfaz — comparación visual	Demo en cuaderno
50 min	Actividad en equipos: construcción del programa	Trabajo en clase
5 min	Cierre: equipos muestran su resultado al grupo	Presentación breve
80 min		

Esta sesión es el puente entre la **teoría** (sesión 11, call stack, recursión) y la **práctica recursiva** (sesión 13, algoritmo que resuelve Hanoi solo). El propósito pedagógico es que los alumnos *sientan* el problema antes de verlo resuelto por la computadora: si lo mueven a mano, la sesión 13 tendrá mucho más impacto cuando la función recursiva lo resuelva en milisegundos.

También es una sesión de **diseño**: los alumnos eligen cómo se ven las torres. Esa diversidad de diseños es insumo para el concurso de la sesión 13. Motivarlos a que el código sea legible desde ahora: el equipo ganador tendrá que explicarlo frente al grupo.

Preguntas de arranque

Lanza estas preguntas antes de abrir el cuaderno:

1. **"¿Cuántos movimientos harían los sacerdotes del templo de Benarés si mueven un disco por segundo, sin parar?"**
   Respuesta esperada: muchos. Guiarlos a que calculen $2^{64} - 1$ y lo conviertan a años: ≈ 585 mil millones de años. El impacto de "42 veces la edad del universo" suele provocar buena reacción.

2. **"¿Cuántos movimientos necesito para mover 1 disco? ¿Y para 2? ¿Y para 3?"**
   Respuesta: 1, 3, 7. Pedir que noten el patrón: siempre $2^n - 1$. No dar la fórmula todavía; dejar que la encuentren ellos.

3. **"¿Qué es más difícil: hacer que el programa funcione, o hacer que se vea bien en la terminal?"**
   No hay respuesta correcta. Sirve para motivar la discusión sobre el valor del diseño visual en las herramientas de consola. Anticipar que algunos alumnos dirán "lo visual no importa si funciona" — usarlo para introducir el argumento de la legibilidad.

Notas de contenido ampliado

- **La historia real detrás del juguete**

Édouard Lucas era un matemático serio: contribuyó a la teoría de números, trabajó con series de Fibonacci y desarrolló pruebas de primalidad. El juguete de Hanoi fue un proyecto secundario, casi una broma comercial, pero se convirtió en uno de los objetos matemáticos más citados de la historia de la computación.

El nombre "d'Hanoï" evoca Indochina francesa (Hanói, Vietnam), que en 1883 era colonia francesa. Lucas eligió el nombre por el exotismo que evocaba, no por alguna conexión cultural real — del mismo modo que inventó la leyenda del templo.

> La historia del templo de Benarés es completamente inventada por Lucas. No existe evidencia de ningún templo, ni de ninguna tradición religiosa que corresponda a esa descripción. Lucas la publicó bajo el seudónimo "N. Claus de Siam" — anagrama de "Lucas d'Amiens" — para darle aire de misterio oriental. Un truco de marketing del siglo XIX.

- **La conexión binaria: más profunda de lo que parece**

La conexión entre Hanoi y el código Gray no es un mero truco matemático: es una manifestación de que el **espacio de estados** del problema de Hanoi tiene exactamente la misma estructura que el número binario de n bits.

Cada estado de las torres se puede codificar como un número: para $n = 3$ hay $3^3 = 27$ estados (cada uno de los 3 discos puede estar en una de las 3 torres). Los $2^3 = 8$ estados del camino óptimo corresponden exactamente a los 8 vértices del cubo de n dimensiones recorrido con el código Gray.

No es necesario explicar esto en clase, pero si algún alumno de matematicas pregunta, esta es la respuesta completa.

- **¿Por qué es recursivo Hanoi por naturaleza?**

El enunciado del problema ya *es* recursivo: para mover n discos, necesitas haber movido $n - 1$ discos. No hay forma de describir la solución sin apelar a la solución del subproblema. Esto es diferente de problemas como la suma de una lista, que pueden describirse iterativamente sin rodeo.

La próxima sesión, cuando los alumnos vean el algoritmo recursivo de 6 líneas, ese "ajá" solo funciona si hoy movieron los discos a mano y sintieron la repetición del patrón.

Cómo conducir la actividad en equipos

> Si el grupo no es múltiplo de 3, es preferible un equipo de 4 que dos equipos de 2. Un equipo de 2 tendrá menos capacidad de dividir el trabajo (dibujo, movimiento, interfaz) y tiende a producir código menos limpio por falta de revisión interna.

- **División de roles dentro del equipo**

Sugerir a los equipos que dividan el trabajo así:

Rol	Responsabilidad
Diseñador	Decide la forma de los discos, el estilo del poste y la base. Trabaja en `dibujarFila` y `dibujarTorres`.
Lógica	Implementa `moverDisco` con las dos validaciones (origen vacío, tamaño). Maneja el estado (`vector<int>`).
Interfaz	Escribe el bucle principal, el prompt al usuario, la detección de victoria, los mensajes de error.

En la práctica los roles se mezclan, pero tenerlos definidos desde el principio evita que los tres alumnos escriban código encima del otro.

• Puntos de control durante los 50 minutos

Min	Qué revisar al pasar por los equipos
15	¿Ya definieron el estado como `vector<int>` con 0 = fondo? ¿Ya pueden imprimir una fila de un disco?
30	¿`dibujarTorres` imprime algo coherente? ¿El disco del tope se dibuja en la fila correcta?
45	¿`moverDisco` rechaza el movimiento ilegal? ¿El bucle pide origen y destino correctamente?
55	¿Se detecta la condición de victoria? ¿El diseño es legible y consistente?

El problema más frecuente es calcular mal cuántos espacios de relleno necesita cada disco. La fórmula que funciona es:

$$\text{espacios} = \frac{\text{anchoTorre} - (2k - 1)}{2}$$

donde k es el número de disco (1 = más pequeño) y `anchoTorre` es el ancho total de la columna (típicamente 9 u 11). Si están usando un carácter diferente a =, la fórmula es la misma: el carácter simplemente reemplaza el =.

Algunos equipos habrán buscado en internet la solución recursiva de Hanoi y querrán implementarla hoy. **No permitirlo.** La actividad de hoy es que el *usuario* haga los movimientos. Si el programa se resuelve solo, el código del equipo no podrá usarse como base para la actividad de la sesión 13 (porque ya la resuelve). Decirles que tendrán esa oportunidad la próxima sesión, y que el reto de hoy es diferente.

FAQs frecuentes de los alumnos

Pregunta: ¿Puedo usar un arreglo `int torre[N]` en lugar de `vector<int>`?
Respuesta: Sí, pero es más incómodo. Con un arreglo necesitas llevar un contador de cuántos discos hay en cada torre. El `vector` ya tiene `size()`, `push_back()` y `pop_back()` que modelan exactamente las operaciones de una pila. Usar un arreglo no está mal, pero sí innecesariamente complicado.

Pregunta: Si la torre A tiene 3 discos y la B tiene 1, ¿cómo imprimo la fila 2 de B (que está vacía)?
Respuesta: La clave es iterar las filas de arriba a abajo (`fila` desde `nFilas-1` hasta 0) y, para cada torre, verificar si el vector tiene un elemento en esa posición. Si `fila < torre.size()`, imprime el disco; si no, imprime el poste vacío (`|` centrado con espacios).

Pregunta: ¿El programa debe soportar cualquier número de discos?
Respuesta: Para la actividad de hoy, con 4 o 5 discos es suficiente. Hardcodear `int N = 4` no es un problema en esta sesión. En la sesión 13 lo parametrizarán si es necesario. Advertir que con muchos discos la terminal se desordena si el ancho no está calculado correctamente.

Pregunta: ¿Imprimo las torres antes del movimiento, después, o las dos?
Respuesta: Mínimo una vez después de cada movimiento. Algunos equipos querrán limpiar la pantalla (`system("clear")`) para que parezca una animación. Está permitido y puede ser un detalle de diseño interesante, pero no es requisito. Si usan `system()`, mencionar que es una llamada al sistema operativo y no funciona igual en Windows (`"cls"`) y en macOS/Linux (`"clear"`).

Pregunta: ¿Tenemos que validar que el input sea A, B o C?
Respuesta: Es una buena práctica, pero no es criterio de evaluación hoy. Si lo implementan, sumarles mérito en diseño visual (el mensaje de error es parte de la interfaz). Un `if (origen != 'A' && origen != 'B' && origen != 'C')` es suficiente.

Evaluación y criterios de calificación

- **Lo que más importa**

Los criterios de evaluación son los de la tabla en las notas del alumno, pero en la práctica el peso real está en dos cosas:

1. **Que el dibujo sea correcto.** Un programa que no muestra las torres de forma coherente no cumple el objetivo fundamental. Si el dibujo falla, no importa qué tan bonito sea el carácter elegido para los discos.
2. **Que la validación rechace discos grandes sobre chicos.** Este es el corazón de las reglas. Un programa que acepta cualquier movimiento es incorrecto por definición.

- ### Criterio de diseño visual — Cómo calibrarlo

El criterio de diseño (20 %) es el más subjetivo. Usar esta escala:

Puntos	Descripción
20	Los discos se distinguen claramente unos de otros, el poste es visible, hay etiquetas A/B/C, la base está bien delimitada, el estilo es consistente (mismo carácter en todos los discos).
15	La torre es legible pero hay inconsistencias menores (e.g., la base no está bien alineada o las etiquetas faltan).
10	Se puede entender qué torre es qué, pero el diseño es mínimo o confuso en algunos estados.
5	El programa imprime algo, pero las torres no son reconocibles como torres.

Conexión con la sesión 13

Al cierre de la sesión, algunos alumnos preguntarán cómo se resuelve Hanoi de forma automática. **No dar la respuesta.** Solamente decir: “la próxima sesión verán que el código es sorprendentemente corto — por eso importa que el código de hoy esté bien estructurado”.

El impacto del algoritmo recursivo de sesión 13 depende del contraste con el esfuerzo que los alumnos pusieron hoy moviendo los discos a mano. Arruinar esa sorpresa reduce el efecto pedagógico.

En la sesión 13:

- Los equipos presentan su programa (30 segundos cada uno, proyectan pantalla o muestran capturas).
- El grupo vota por los tres diseños más atractivos con código legible.
- El equipo ganador comparte su código en pantalla.
- Se agrega la función recursiva `hanoi(n, origen, destino, aux)` al código ganador.
- Se ejecuta la solución completa y se verifica que el número de movimientos sea $2^n - 1$.

Para que la sesión 13 funcione bien, es importante que:

1. Los equipos hayan subido su código a Canvas **antes** de que termine la sesión 12 (o como máximo al inicio de la sesión 13).
2. El código ganador esté disponible en pantalla desde el inicio.
3. Las tres torres estén representadas como `vector<int>` (o algo equivalente como pilas), de modo que `moverDisco` pueda ser llamada desde la función recursiva sin reescribir.

Soluciones a los ejercicios a mano

El estado de las torres en cada paso (A, B, C) — lista de discos de abajo a arriba, siendo 3 el más grande y 1 el más pequeño:

Estado	A	B	C
0 (inicial)	[3,2,1]	[]	[]
1	[3,2]	[]	[1]
2	[3]	[2]	[1]
3	[3]	[2,1]	[]
4	[]	[2,1]	[3]
5	[1]	[2]	[3]
6	[1]	[]	[3,2]
7 (final)	[]	[]	[3,2,1]

Verificación: $M(3) = 2^3 - 1 = 7$. ✓

Usando la recurrencia $M(n) = 2\,M(n-1) + 1$:

$$M(4) = 2 \cdot 7 + 1 = 15 \qquad M(5) = 2 \cdot 15 + 1 = 31$$

Verificación directa: $2^4 - 1 = 15$, $2^5 - 1 = 31$. ✓

La función recursiva se llama a sí misma con $n-1$, $n-2$, …, 1. La profundidad máxima es $n = 4$ **frames** activos simultáneamente en el stack (más el frame de `main`). La profundidad del árbol de recursión es n, no $2^n - 1$ (que es el número de hojas / llamadas totales).

Cuando k es potencia de 2, su representación binaria tiene exactamente un bit en 1: el bit más significativo. El bit *menos* significativo diferente de cero coincide con ese mismo bit. Por ejemplo: $k = 4 = 100_2 \to$ bit 2 $\to$ disco 3 (el más grande de la secuencia óptima de $n = 3$). En general, cuando $k = 2^j$, se mueve el disco

$j + 1$.

Con 4 agujas se necesitan **menos** movimientos. La intuición: la cuarta aguja da un destino temporal adicional, lo que permite mover grupos de discos en lugar de uno a la vez. El resultado exacto lo dio Frame en 1941: para n discos y 4 agujas, el mínimo aproximado es $\Theta(2\sqrt{n})$ movimientos, mucho menor que 2^n. (No es necesario que los alumnos lleguen a la fórmula — basta la intuición de "menos movimientos porque hay más opciones intermedias".)

Resumen para el profesor

✔ Recuerda

- La actividad central son 50 minutos de trabajo en equipos de tres.
- El objetivo real no es el código: es que los alumnos *sientan* el problema antes de verlo resuelto automáticamente en la sesión 13.
- No revelar el algoritmo recursivo — la sorpresa tiene valor pedagógico.
- Verificar al minuto 45 que todos los equipos tengan la validación de movimiento ilegal funcionando.
- Los equipos suben código y capturas a Canvas al final de la sesión.